ANASTASIA LABS

Proof of Achievement - Milestone 2

Augmenting Lucid's Utility Library Functions

Project Number 1100024
Project Manager Jonathan Rodriguez

ANASTASIA LABS

Contents

Outline Report for Utility Functions per Package 1
bip39.ts 2
address.ts 3
cbor.ts 4
cost_model.ts 5
credential.ts 6
datum.ts 7
keys.ts 8
native.ts 9
network.ts 10
scripts.ts 11
time.ts 12
utxo.ts 13
value.ts 14

Project Name: Lucid Evolution: Redefining Off-Chain Transactions in Cardano
URL: Catalyst Proposal

Outline Report for Utility Functions per Package

In this following section you will find information about each utility package implemented in our project. These modules contain functions that are designed versatile and are to help reduce redundancy. These packages are from the lucid-evolution repository with the following format:

1. Title
2. Description
3. Key Functions
4. Code Snapshot

By clicking on the "GitHub Link" hyperlink you can view the dedicated repository section for the utility function package

Description

The `bip39.ts` module implements functions related to BIP39, which defines a way to generate the mnemonic phrase (a series of easy-to-remember words) from a random seed

- This is a partial reimplementation of BIP39 in Deno
- We only use the default Wordlist (english)

Utility Package	Directory
bip39.ts	GitHub Link

Key Functions

`mnemonicToEntropy`

Converts a mnemonic phrase back into its original entropy representation

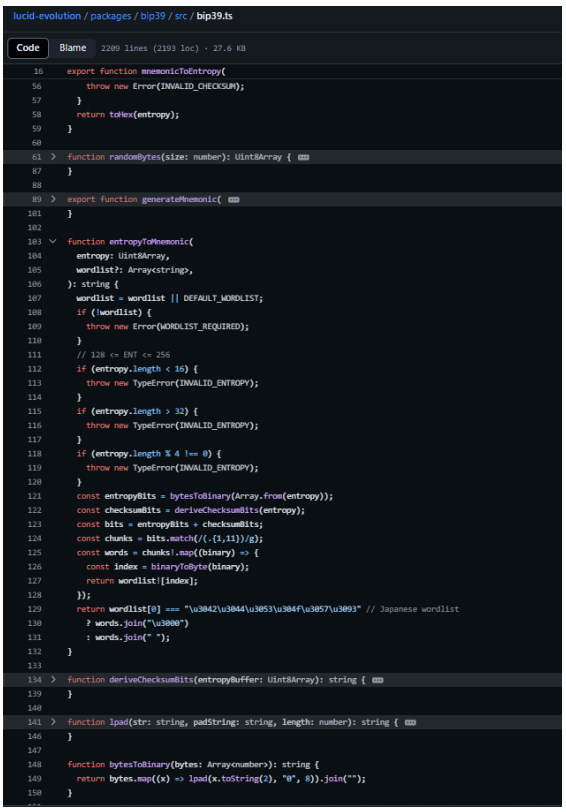
`generateMnemonic`

Generates a new mnemonic phrase from random entropy. It can be used to create new wallets or regenerate existing ones from a known entropy source

`entropyToMnemonic`

Converts entropy into a mnemonic phrase using a specific wordlist for wallet recovery or setup

Code Snapshot



lucid-evolution / packages / bip39 / src / bip39.ts

Code Blame 2289 lines (2193 loc) · 27.6 KB

```
16  export function mnemonicToEntropy(
56      throw new Error(INVALID_CHECKSUM);
57    }
58    return toHex(entropy);
59  }
60
61  function randomBytes(size: number): Uint8Array { ...
87  }
88
89  export function generateMnemonic( ...
101 }
102
103 function entropyToMnemonic(
104   entropy: Uint8Array,
105   wordlist?: Array<string>,
106 ): string {
107   wordlist = wordlist || DEFAULT_WORDLIST;
108   if (!wordlist) {
109     throw new Error(WORDLIST_REQUIRED);
110   }
111   // 128 <= ENT <= 256
112   if (entropy.length < 16) {
113     throw new TypeError(INVALID_ENTROPY);
114   }
115   if (entropy.length > 32) {
116     throw new TypeError(INVALID_ENTROPY);
117   }
118   if (entropy.length % 4 !== 0) {
119     throw new TypeError(INVALID_ENTROPY);
120   }
121   const entropyBits = bytesToBinary(Array.from(entropy));
122   const checksumBits = deriveChecksumBits(entropy);
123   const bits = entropyBits + checksumBits;
124   const chunks = bits.match(/(.{1,11})/g);
125   const words = chunks!.map((binary) => {
126     const index = binaryToByte(binary);
127     return wordlist![index];
128   });
129   return wordlist[0] === "\u3042\u3044\u3053\u304f\u3057\u3093" // Japanese wordlist
130     ? words.join("\u3000")
131     : words.join(" ");
132 }
133
134 function deriveChecksumBits(entropyBuffer: Uint8Array): string { ...
139 }
140
141 function lpad(str: string, padString: string, length: number): string { ...
146 }
147
148 function bytesToBinary(bytes: Array<number>): string {
149   return bytes.map((x) => lpad(x.toString(2), "0", 8)).join("");
150 }
```

Figure 1: Snapshot-01-BIP39

Description

The `address.ts` module is used to handle address-related operations within the Lucid Evolution library. Its functions allow the manipulation and conversion of various address types

Utility Package	Directory
address.ts	GitHub Link

Key Functions

`addressFromHexOrBech32`

Converts an address from either hexadecimal or Bech32 format to a CML Address object

`credentialToRewardAddress`

Converts a stake credential into a reward address

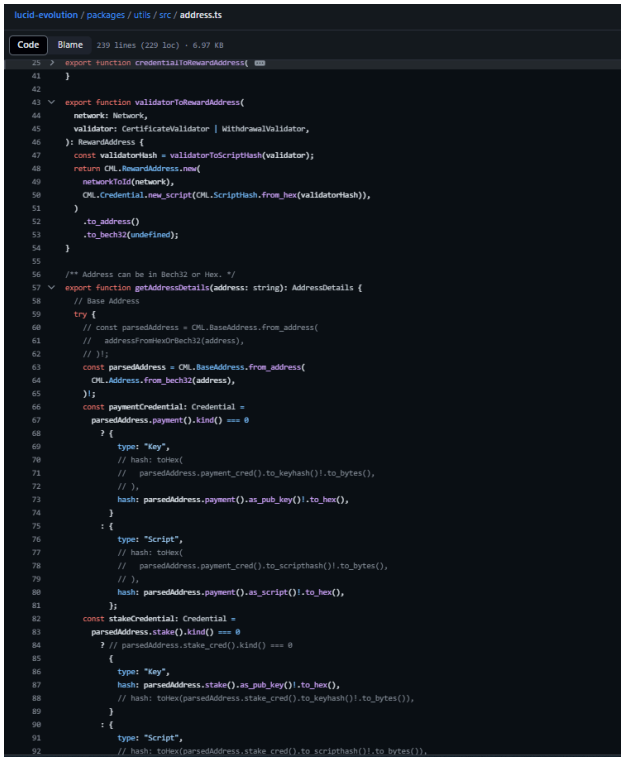
`validatorToRewardAddress`

Converts a validator (either a certificate or withdrawal validator) into a reward address using the script hash derived from the validator

`getAddressDetails`

Extracts and returns detailed information about various address types (Base, Enterprise, Pointer, Reward, Byron), including payment and stake credentials

Code Snapshot


lucid-evolution / packages / utils / src / address.ts

Code Blame 239 lines (229 loc) · 6.97 KB

```
25  export function credentialToRewardAddress(
41  }
42
43  export function validatorToRewardAddress(
44    network: Network,
45    validator: CertificateValidator | WithdrawalValidator,
46  ): RewardAddress {
47    const validatorHash = validatorToScriptHash(validator);
48    return CML.RewardAddress.new(
49      networkToId(network),
50      CML.Credential.new_script(CML.ScriptHash.from_hex(validatorHash)),
51    )
52      .to_address()
53      .to_bech32(undefined);
54  }
55
56  /** Address can be in Bech32 or Hex. */
57  export function getAddressDetails(address: string): AddressDetails {
58    // Base Address
59    try {
60      // const parsedAddress = CML.BaseAddress.from_address(
61      //   addressFromHexOrBech32(address),
62      // );
63      const parsedAddress = CML.BaseAddress.from_address(
64        CML.Address.from_bech32(address),
65      );
66      const paymentCredential: Credential =
67        parsedAddress.payment().kind() === 0
68          ? {
69              type: "Key",
70              // hash: toHex(
71              //   parsedAddress.payment_cred().to_keyhash()!.to_bytes(),
72              // ),
73              hash: parsedAddress.payment().as_pub_key()!.to_hex(),
74            }
75          : {
76              type: "Script",
77              // hash: toHex(
78              //   parsedAddress.payment_cred().to_scripthash()!.to_bytes(),
79              // ),
80              hash: parsedAddress.payment().as_script()!.to_hex(),
81            };
82      const stakeCredential: Credential =
83        parsedAddress.stake().kind() === 0
84          ? // parsedAddress.stake_cred().kind() === 0
85            {
86              type: "Key",
87              hash: parsedAddress.stake().as_pub_key()!.to_hex(),
88              // hash: toHex(parsedAddress.stake_cred().to_keyhash()!.to_bytes()),
89            }
90          : {
91              type: "Script",
92              // hash: toHex(parsedAddress.stake_cred().to_scripthash()!.to_bytes()),
```

Figure 2: Snapshot-02-Address

Description

The cbor.ts module within Lucid Evolution deals with functionalities related to CBOR (Concise Binary Object Representation), specifically focusing on encoding and decoding operations that adhere to the CBOR standard as defined in RFC 7049

Utility Package	Directory
cbor.ts	GitHub Link

Key Functions

`applyDoubleCborEncoding`

Implements a double encoding for CBOR bytestrings, which decodes an encoded string twice to ensure correct formatting

Code Snapshot

lucid-evolution / packages / utils / src / cbor.ts

solidsnakedev feat: update applyDoubleCborEncoding

Code Blame 16 lines (15 loc) · 568 Bytes

```
import { fromHex, toHex } from "@lucid-evolution/core-utils";
import { decode, encode } from "cborg";

// 1st byte (58) 0101(major type 2) , 1000 (additional info)
// 2n byte byte represents the lenght of the content
// 3rd byte represents bytestring content
// https://www.rfc-editor.org/rfc/rfc7049#section-2.1
// Apply double bytestring enconding of type `major type 2`
export const applyDoubleCborEncoding = (script: string) => {
  try {
    decode(decode(fromHex(script)));
    return script;
  } catch (error) {
    return toHex(encode(fromHex(script)));
  }
};
```

Figure 3: Snapshot-03-Cbor

Description

The `cost_model.ts` module in Lucid Evolution deals with the configuration and management of cost models related to the execution of Plutus scripts on the blockchain. These cost models are used to in determine the computational and memory costs of running smart contracts

Utility Package	Directory
cost_model.ts	GitHub Link

Key Functions

`createCostModels`

Constructs a CostModels object that covers the various cost parameters for different versions of the Plutus scripts (PlutusV1, PlutusV2).

This function populates cost models from predefined settings.

1. Initializes new cost model objects for each Plutus version
2. Iteratively fills these objects with cost data parsed from input parameters
3. Handles the memory management of these operations to prevent leaks and ensure efficiency

Code Snapshot

lucid-evolution / packages / utils / src / cost_model.ts

Code Blame 398 lines (392 loc) · 17.4 KB

```
4   export function createCostModels(costModels: CostModels): CML.CostModels {
5     const costmdls = CML.CostModels.new();
6
7     // add plutus v1
8     const costmdlV1 = CML.IntList.new();
9     for (const cost of Object.values(costModels.PlutusV1)) {
10      const int = CML.Int.from_str(cost.toString());
11      costmdlV1.add(int);
12      int.free();
13    }
14    costmdls.set_plutus_v1(costmdlV1);
15    costmdlV1.free();
16
17    // add plutus v2
18    const costmdlV2 = CML.IntList.new();
19    for (const cost of Object.values(costModels.PlutusV2)) {
20      const int = CML.Int.from_str(cost.toString());
21      costmdlV2.add(int);
22      int.free();
23    }
24    costmdls.set_plutus_v2(costmdlV2);
25    costmdlV2.free();
26
27    // add plutus v3
28    // const costmdlV3 = C.IntList.new()
29    // Object.values(costModels.PlutusV3).forEach((cost) => {
30    //   costmdlV3.add(C.Int.new(BigInt(cost)))
31    // });
32    // costmdls.set_plutus_v3(costmdlV3)
33
34    return costmdls;
35  }
36
37  export const PROTOCOL_PARAMETERS_DEFAULT: ProtocolParameters = {
38    minFeeA: 44,
39    minFeeB: 155381,
40    maxTxSize: 16384,
41    maxValSize: 5000,
42    keyDeposit: 2000000n,
43    poolDeposit: 500000000n,
44    priceMem: 0.0577,
45    priceStep: 0.0000721,
46    maxTxExMem: 14000000n,
47    maxTxExSteps: 10000000000n,
48    coinsPerUtxoByte: 4310n,
49    collateralPercentage: 150,
50    maxCollateralInputs: 3,
51    costModels: {
52      PlutusV1: {
53        "addInteger-cpu-arguments-intercept": 205665,
54        "addInteger-cpu-arguments-slope": 812,
55        "addInteger-memory-arguments-intercept": 1,
```

Figure 4: Snapshot-04-Costmodel

Description

The `credential.ts` module handles the creation and manipulation of credentials within the ecosystem. This module is for constructing addresses and managing their components.

Utility Package	Directory
credential.ts	GitHub Link

Key Functions

`credentialToAddress`
Converts payment and optionally stake credentials into an address

`scriptHashToCredential`
Wraps a script hash into a credential object, utilizing its use in other functions requiring a credential format

`keyHashToCredential`
Converts a key hash into a credential object, allowing for further operations that require credentials

`paymentCredentialOf`
Extracts the payment credential from an address, throwing an error if the address does not contain one

`stakeCredentialOf`
Retrieves the stake credential from a reward address

Code Snapshot

lucid-evolution / packages / utils / src / credential.ts

Code Blame 86 lines (81 loc) · 2.25 KB

```
  ScriptHash,
  KeyHash,
  RewardAddress,
} from "@lucid-evolution/core-types";
import { networkToId } from "./network.js";
import { CML } from "./core.js";
import { getAddressDetails } from "./address.js";

export function credentialToAddress(
  network: Network,
  paymentCredential: Credential,
  stakeCredential?: Credential,
): Address {
  if (stakeCredential) {
    return CML.BaseAddress.new(
      networkToId(network),
      paymentCredential.type === "Key"
        ? CML.Credential.new_pub_key(
            CML.Ed25519KeyHash.from_hex(paymentCredential.hash),
          )
        : CML.Credential.new_script(
            CML.ScriptHash.from_hex(paymentCredential.hash),
          ),
      stakeCredential.type === "Key"
        ? CML.Credential.new_pub_key(
            CML.Ed25519KeyHash.from_hex(stakeCredential.hash),
          )
        : CML.Credential.new_script(
            CML.ScriptHash.from_hex(stakeCredential.hash),
          ),
    )
      .to_address()
      .to_bech32(undefined);
  } else {
    return CML.EnterpriseAddress.new(
      networkToId(network),
      paymentCredential.type === "Key"
        ? CML.Credential.new_pub_key(
            CML.Ed25519KeyHash.from_hex(paymentCredential.hash),
          )
        : CML.Credential.new_script(
            CML.ScriptHash.from_hex(paymentCredential.hash),
          ),
    )
      .to_address()
      .to_bech32(undefined);
  }
}

export function scriptHashToCredential(scriptHash: ScriptHash): Credential {
  return {
    type: "Script",
    hash: scriptHash,
```

Figure 5: Snapshot-05-Credential

Description

The `datum.ts` module provides functionality for handling Plutus data on the blockchain. Specifically, it includes utilities for converting Plutus data (datum) into a format that is suitable for transaction processing, like generating a hash of the datum

Utility Package	Directory
datum.ts	GitHub Link

Key Functions

`datumToHash`

Converts a datum object into its corresponding hash. This hash is used to refer to data stored off-chain.

1. Converts the datum from its CBOR hexadecimal representation to a Plutus data format
2. Uses the CML to calculate the hash of the Plutus data

Code Snapshot

lucid-evolution / packages / utils / src / **datum.ts**

solidsnakedev refactor: move CML to core file

Code Blame 6 lines (5 loc) · 232 Bytes

```
import { Datum, DatumHash } from "@lucid-evolution/core-types";
import { CML } from "./core.js";

export function datumToHash(datum: Datum): DatumHash {
  return CML.hash_plutus_data(CML.PlutusData.from_cbor_hex(datum)).to_hex();
}
```

Figure 6: Snapshot-06-Datum

Description

The `keys.ts` deals with the generation and conversion of keys which are fundamental for secure transactions

Utility Package	Directory
keys.ts	GitHub Link

Key Functions

`generatePrivateKey`

Generates a new private key using the ED25519 cryptographic algorithm. This key is used for signing transactions securely

`generateSeedPhrase`

Creates a mnemonic seed phrase based on the BIP39 standard

`toPublicKey`

Converts a given private key to its corresponding public key, allowing for the public key to be used in transaction verification without revealing the private key

An example of a key private -> public conversion would look like:

```
CML.PrivateKey.from_bech32(privateKey).to_public().to_bech32();
```

Code Snapshot

Figure 7: Snapshot-07-Keys

Description

The `native.ts` module handles operations related to Cardano's native scripts, which are used for transaction validation without the execution of Plutus smart contracts. This module provides functionality to convert custom native script objects into Cardano's native script format

Utility Package	Directory
native.ts	GitHub Link

Key Functions

`toNativeScript`

Converts a high-level native script definition into a low-level script that the Cardano node can interpret. This function supports various types of native scripts including simple public key-based scripts, time-lock scripts, and complex multi-script conditions

`nativeJSFromJson`

Encapsulates the conversion of a Native script object into a script that is compatible with the ledger, serialized into CBOR hex format

Code Snapshot

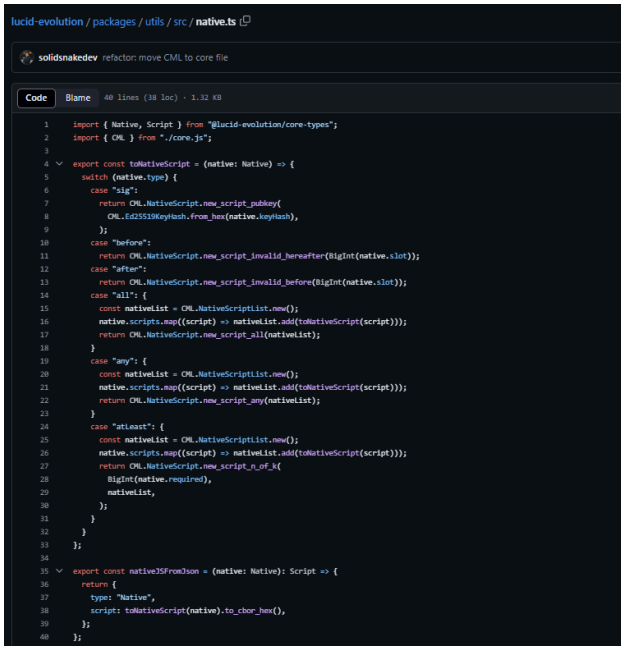


Figure 8: Snapshot-08-Native

Description

The `network.ts` module is to map high-level network identifiers to their corresponding numeric identifiers

Utility Package	Directory
network.ts	GitHub Link

Key Functions

`networkToId`

Converts a network name into its corresponding numeric ID

Mapping process

```
export function networkToId(network: Network): number {
  switch (network) {
    case "Preview":
      return 0;
    case "Preprod":
      return 0;
    case "Custom":
      return 0;
    case "Mainnet":
      return 1;
    default:
      throw new Error("Network not found");
  }
}
```

This function's purpose is to ensure that transactions are correctly associated with the appropriate network

Description

The `scripts.ts` module offers a range of functions to manage and transform scripts used in smart contracts. It handles various script types including native, Plutus V1, and Plutus V2 scripts, facilitating their usage in transactions and smart contracts

Utility Package	Directory
scripts.ts	GitHub Link

Key Functions

`validatorToAddress`

Converts a validator script into a Cardano address

`validatorToScriptHash`

Generates a script hash from a validator object. This function supports multiple script types including Native, Plutus V1, and Plutus V2

`toScriptRef / fromScriptRef`

Converts a script into a CML.Script object and vice versa, facilitating the use of scripts in a format suitable for transactions

`mintingPolicyToId`

Converts a minting policy into a policy ID using the script hash functionality

`nativeFromJson / nativeScriptFromJson`

Converts JSON representations of native scripts into script objects, so that scripts can be handled in a standardized format across the system

`applyParamsToScript`

Applies parameters to a Plutus script

Code Snapshot

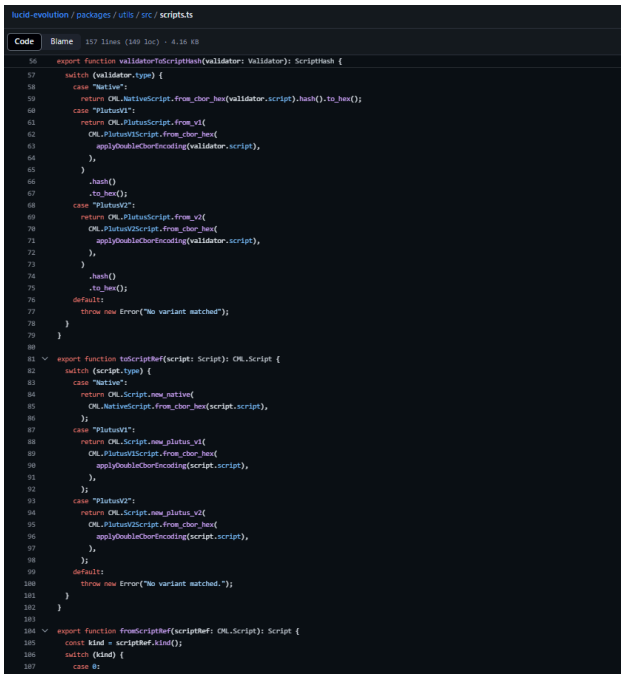


Figure 9: Snapshot-09-Scripts

Description

The `time.ts` module in our library handles the conversion between blockchain-specific slot numbers and Unix timestamps. This functionality is important for scheduling and timing events within the blockchain, where time is often expressed in terms of slots

Utility Package	Directory
time.ts	GitHub Link

Key Functions

`unixTimeToSlot`
Converts a Unix timestamp to the corresponding slot number in the blockchain. It is to determine when specific events or transactions should occur relative to blockchain time

`slotToUnixTime`
Converts a slot number to the corresponding Unix timestamp. This allows applications to interpret blockchain time in terms of real-world time

What are slots and how do they serve a role in time?

These functions use `SLOT_CONFIG_NETWORK`, a predefined mapping specific to each network configuration that defines the relationship between slot numbers and Unix time. This ensures accurate time calculations across different network settings

```
export function slotToUnixTime(network: Network, slot: Slot): UnixTime {
  return slotToBeginUnixTime(slot, SLOT_CONFIG_NETWORK[network]);
}
```

Description

The `utxo.ts` module provides functionality for managing UTxOs. It supports creating transaction inputs and outputs, converting UTxOs to different formats, and sorting or selecting UTxOs based on specific criteria

Utility Package	Directory
utxo.ts	GitHub Link

Key Functions

`utxoToTransactionOutput` / `utxoToTransactionInput`

These functions convert UTxO data into transaction outputs and inputs, facilitating the integration of UTxOs into new transactions

`utxoToCore` / `utxosToCores`

Converts UTxOs to CML.TransactionUnspentOutput objects, standardizing UTxOs for transaction processing

`coreToUtxo` / `coresToUtxos`

Reverses the conversion process, transforming CML.TransactionUnspentOutput objects back into UTxO format

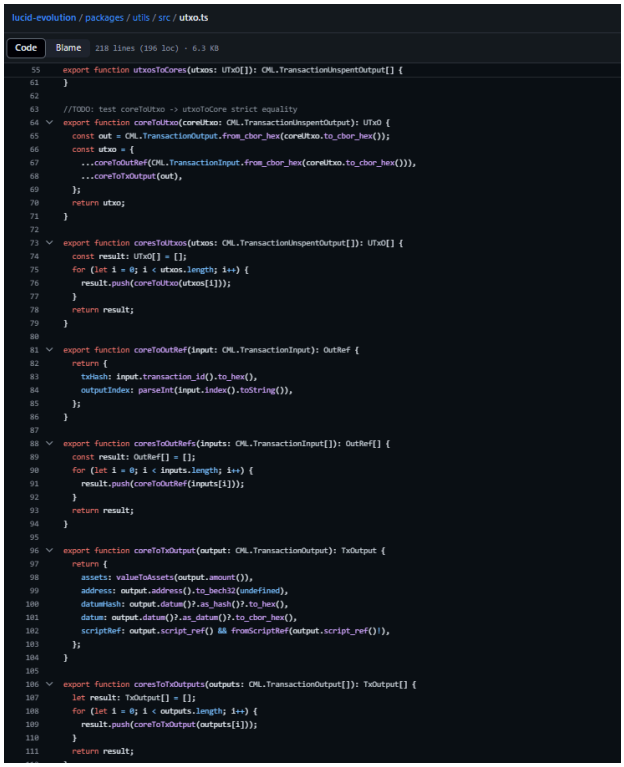
`selectUTxOs`

Selects UTxOs from a list that meet specified asset requirements, useful in transaction construction where specific asset amounts are required

`sortUTxOs`

Sorts an array of UTxOs according to a specified order, either largest first or smallest first, based on the amount of Lovelace

Code Snapshot



lucid-evolution / packages / utils / src / utxo.ts

Code Blame 218 lines (196 loc) · 6.3 KB

```
55  export function utxosToCores(utxos: UTxO[]): CML.TransactionUnspentOutput[] {
61  }
62
63  //TODO: test coreToUtxo -> utxoToCore strict equality
64  export function coreToUtxo(coreUtxo: CML.TransactionUnspentOutput): UTxO {
65    const out = CML.TransactionOutput.from_cbor_hex(coreUtxo.to_cbor_hex());
66    const utxo = {
67      ...coreToOutRef(CML.TransactionInput.from_cbor_hex(coreUtxo.to_cbor_hex())),
68      ...coreToTxOutput(out),
69    };
70    return utxo;
71  }
72
73  export function coresToUtxos(utxos: CML.TransactionUnspentOutput[]): UTxO[] {
74    const result: UTxO[] = [];
75    for (let i = 0; i < utxos.length; i++) {
76      result.push(coreToUtxo(utxos[i]));
77    }
78    return result;
79  }
80
81  export function coreToOutRef(input: CML.TransactionInput): OutRef {
82    return {
83      txHash: input.transaction_id().to_hex(),
84      outputIndex: parseInt(input.index().toString()),
85    };
86  }
87
88  export function coresToOutRefs(inputs: CML.TransactionInput[]): OutRef[] {
89    const result: OutRef[] = [];
90    for (let i = 0; i < inputs.length; i++) {
91      result.push(coreToOutRef(inputs[i]));
92    }
93    return result;
94  }
95
96  export function coreToTxOutput(output: CML.TransactionOutput): TxOutput {
97    return {
98      assets: valueToAssets(output.amount()),
99      address: output.address().to_bech32(undefined),
100     datumHash: output.datum()?.as_hash()?.to_hex(),
101     datum: output.datum()?.as_datum()?.to_cbor_hex(),
102     scriptRef: output.script_ref() && fromScriptRef(output.script_ref()!),
103   };
104 }
105
106 export function coresToTxOutputs(outputs: CML.TransactionOutput[]): TxOutput[] {
107   let result: TxOutput[] = [];
108   for (let i = 0; i < outputs.length; i++) {
109     result.push(coreToTxOutput(outputs[i]));
110   }
111   return result;
```

Figure 10: Snapshot-10-Utxo

Description

The `value.ts` module provides functions to manipulate and convert between the blockchain's internal value representation and a more accessible assets format. This serves the purpose for managing transaction outputs and state transitions in smart contracts

Utility Package	Directory
value.ts	GitHub Link

Key Functions

`valueToAssets`

Converts a CML.Value object, which represents the amount of different tokens in a transaction output, into an Assets object that is easier to manipulate and display

`assetsToValue`

Converts an Assets object back into a CML.-Value object for use in transaction creation or other on-chain activities

`fromUnit` / `toUnit`

These functions handle conversion between a unit representation (combining policy ID and asset names) and its constituent parts, helping in asset identification and manipulation

`addAssets`

Aggregates multiple Assets objects into a single object, summing up quantities of the same assets

Code Snapshot

lucid-evolution / packages / utils / src / value.ts

Code Blame 101 lines (96 loc) · 3.21 KB

```
      const assetNames = policyAssets.keys();
      for (let k = 0; k < assetNames.len(); k++) {
        const policyAsset = assetNames.get(k);
        const quantity = policyAssets.get(policyAsset)!;
        //FIX: report to dcspark policyAsset.to_cbor_hex() adds the head byte twice eg. MyMintedToken -> (to Hex) -> 4d4d794d6f
        const unit = policy.to_hex() + fromText(policyAsset.to_str());
        assets[unit] = quantity;
      }
    }
  }
  return assets;
}

export function assetsToValue(assets: Assets): CML.Value {
  const multiAsset = CML.MultiAsset.new();
  const lovelace = assets["lovelace"] ? assets["lovelace"] : 0n;
  const units = Object.keys(assets);
  const policies = Array.from(
    new Set(
      units
        .filter((unit) => unit !== "lovelace")
        .map((unit) => unit.slice(0, 56)),
    ),
  );
  for (const policy of policies) {
    const policyUnits = units.filter((unit) => unit.slice(0, 56) === policy);
    const assetsValue = CML.MapAssetNameToCoin.new();
    for (const unit of policyUnits) {
      assetsValue.insert(
        CML.AssetName.from_bytes(fromHex(unit.slice(56))),
        BigInt(assets[unit]),
      );
    }
    multiAsset.insert_assets(CML.ScriptHash.from_hex(policy), assetsValue);
  }
  return CML.Value.new(lovelace, multiAsset);
}

/**
 * Splits unit into policy id, asset name (entire asset name), name (asset name without label) and label if applicable.
 * name will be returned in Hex.
 */
export function fromUnit(unit: Unit): {
  policyId: PolicyId;
  assetName: string | null;
  name: string | null;
  label: number | null;
} {
  const policyId = unit.slice(0, 56);
  const assetName = unit.slice(56) || null;
  const label = fromLabel(unit.slice(56, 64));
  const name = (() => {
```

Figure 11: Snapshot-11-Value